Modeling Telecom Customer Behavior using Deep Generative Models for Churn Prediction and Data Augmentation

Padma Bandi, Varshini Sherapur Basavarajappa, Abhaya Basnet
pbandi@depaul.edu, vsherapu@depaul.edu, abasnet@depaul.edu

November 21, 2025

Abstract

We investigate telecom customer churn prediction using deep generative models compared to traditional machine learning. Using the IBM Telecom dataset (7,043 customers, 26.5% churn rate), we implement VAEs for learning latent representations and GANs for synthetic data generation to address class imbalance. After extensive optimization, reducing VAE loss by 23% and stabilizing GAN training, traditional methods still outperform deep learning. SMOTE with Logistic Regression achieves the best F1 score (0.6265), surpassing VAE-enhanced features (0.5758) and GAN synthetic data (0.5881). While deep models successfully learn data distributions, they fail to discover patterns beyond well-engineered features. This provides empirical evidence that model complexity should match data characteristics.

1 Introduction

Customer churn directly impacts telecom revenue, with retention costing 5-25x less than acquisition. The ability to identify at-risk customers enables targeted interventions through personalized offers and improved support. The churn prediction problem presents several challenges: class imbalance (typically 20-30% churn rate), heterogeneous features mixing demographics and usage patterns, and the need for interpretable insights beyond predictions. Traditional approaches using Logistic Regression and Random Forests work reasonably well but depend on labeled data and provide limited behavioral insights. SMOTE addresses imbalance through interpolation but may create unrealistic samples. Deep generative models offer potential solutions. VAEs can discover latent customer representations beyond hand-crafted features, potentially revealing hidden behavioral patterns. GANs can generate more realistic synthetic data generation by learning the true data distribution rather than simple interpolation. However, their effectiveness on structured data is not very clear. We implement and optimize both VAEs and GANs for telecom churn, comparing performance against traditional baselines. Our contributions include: implementation for tabular data, hyperparameter optimization, empirical comparison, and analysis of why deep learning underperforms on this task.

2 Related Work

Customer churn prediction has traditionally relied on supervised learning methods such as Logistic Regression, Decision Trees, and Gradient Boosting [6, 4]. While effective for classification, these discriminative models depend heavily on labeled data and often struggle with class imbalance and limited interpretability. They can predict which customers may churn but offer little insight into why such behavior occurs.

Recent work has explored using **deep generative models** to overcome these challenges. Variational Autoencoders (VAEs) [5] and Generative Adversarial Networks (GANs) [3] learn the underlying data distribution, enabling synthetic data generation and latent feature discovery. For instance, CTGAN-ENN, a hybrid approach combining tabular GANs with edited nearest neighbors, has shown strong results for handling imbalanced and overlapping churn data [1]. Similarly, hybrid architectures like VAE-GANs have shown the ability to generate realistic synthetic tabular data by capturing both continuous and categorical feature relationships [2].

3 Preliminary/Background

3.1 Problem Formulation

Given customer feature vector $\mathbf{x} \in \mathbb{R}^d$ and binary churn label $y \in \{0, 1\}$, we aim to learn function $f : \mathcal{X} \to \mathcal{Y}$ maximizing prediction accuracy while handling class imbalance. The dataset exhibits $P(y = 1) \approx 0.265$, meaning about 26% of customers churn. This means about 73.5% do not churn, a clear class imabalance.

3.2 Variational Autoencoders

VAEs learn probabilistic mapping between data $\mathbf{x}$ and latent code $\mathbf{z}$ through encoder $q_\phi(\mathbf{z}|\mathbf{x})$ and decoder $p_\theta(\mathbf{x}|\mathbf{z})$. The objective maximizes the evidence lower bound (ELBO):

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{KL}[q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z})] \tag{1}$$

where the first term ensures reconstruction quality and KL divergence regularizes the latent space. The reparameterization trick enables backpropagation:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}) \tag{2}$$

3.3 Generative Adversarial Networks

GANs train generator G and discriminator D adversarially. G transforms noise $\mathbf{z} \sim p(\mathbf{z})$ into synthetic samples $G(\mathbf{z})$. D distinguishes real from fake. The minimax objective:

$$\min_G \max_D \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}[\log(1 - D(G(\mathbf{z})))] \tag{3}$$

For conditional generation with class labels c, both networks receive label information: $G(\mathbf{z}, c)$ and $D(\mathbf{x}, c)$.

3.4 SMOTE Baseline

SMOTE generates synthetic minority samples through interpolation. For minority sample $\mathbf{x}_i$, it finds k nearest neighbors and creates synthetic sample:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + \lambda(\mathbf{x}_j - \mathbf{x}_i), \quad \mathbf{x}_j \in \text{NN}_k(\mathbf{x}_i), \quad \lambda \sim \mathcal{U}(0, 1) \tag{4}$$

3.5 Preliminary Analysis

Initial data exploration revealed: 26.5% churn rate creating significant imbalance, strong predictors including contract type and tenure, and mixed feature types requiring careful pre-processing. These insights informed our methodology choices.

4 Methodology

4.1 Data Pipeline

We processed the IBM Telecom dataset through three main stages. First, we cleaned the data by converting TotalCharges from string to numeric format and filling 11 missing values with the corresponding MonthlyCharges, assuming new customers hadn't been billed yet.

For feature engineering, we created several relevant features. AvgChargesPerTenure captures how much customers pay per month on average throughout their relationship. CustomerValue represents lifetime value by multiplying tenure with monthly charges. NumAddOnServices counts how many optional services like OnlineBackup or StreamingTV each customer uses. We also converted contract types into ordinal values: month-to-month as 0, one-year as 1, and two-year as 2, reflecting different commitment levels.

The encoding stage converted all categorical variables to numbers. Binary features like PhoneService became 1 for "Yes" and 0 for "No". Multi-category variables like PaymentMethod were one-hot encoded, creating separate columns for each option. This process expanded our features from 21 to 34 total.

4.2 VAE Implementation

Our Variational Autoencoder learns to compress customer profiles into a smaller representation that captures essential patterns. Think of it like creating a summary that preserves the most important information while removing noise.

The architecture consists of an encoder that compresses 34 features down to 20 latent dimensions, and a decoder that reconstructs the original features. We chose 20 latent dimensions after testing 10, 16, and 20, finding that 10 lost too much information while 20 preserved important patterns.

During training, the model learns by trying to reconstruct customer profiles while keeping the latent space well-organized. We balance two goals: accurate reconstruction and maintaining smooth latent space structure. The beta parameter ($\beta = 0.1$) controls this trade-off, prioritizing reconstruction quality.

Training ran for 500 epochs using the Adam optimizer with learning rate 0.001. The loss decreased from 25.6 to 17.0, a 23% improvement, indicating the model successfully learned to compress and reconstruct customer data. This improvement came after careful tuning of the hyper-parameters and increasing the number of training iterations from 150 to 500 epochs.

4.3 GAN Training

Our Generative Adversarial Network creates synthetic churned customers to balance the dataset. Unlike SMOTE which creates customers between existing ones, GANs learn what makes a churned customer and generate entirely new examples.

The generator takes random noise plus a churn label and produces synthetic customer profiles. The discriminator tries to distinguish real customers from generated ones. These networks compete, the generator tries to fool the discriminator while the discriminator gets better at detecting fakes.

Initial training failed with the discriminator becoming perfect at spotting fakes, causing the generator to give up. We stabilized training through three key modifications. Label smoothing used 0.9 and 0.1 instead of 1 and 0 targets, preventing overconfidence. We trained the discriminator only half as often, giving the generator time to improve. Adding small random noise to inputs prevented the discriminator from memorizing exact patterns.

These changes achieved balanced training with discriminator loss stabilizing around 0.72 and generator loss around 1.87, indicating healthier (although not ideal) adversarial dynamics without mode collapse or either network dominating.

4.4 Model Evaluation

We compared five different data approaches to understand which methods work best. The original approach used imbalanced data as-is. SMOTE created synthetic samples through interpolation to balance classes. VAE-latent used only the compressed 20-dimensional representations. VAE-combined used both original and latent features. GAN-synthetic balanced classes using GAN-generated samples.

Each approach was tested with multiple classifiers including Logistic Regression, Random Forest, and XGBoost. We used 80% of data for training and 20% for testing, with stratified sampling to maintain class proportions. Five-fold cross-validation ensured results weren't dependent on a particular data split.

5 Numerical Experiments

5.1 Performance Results

Our experiments revealed surprising results that challenge assumptions about deep learning superiority. SMOTE with Logistic Regression achieved the best F1 score of 0.627, significantly outperforming all deep learning approaches.

The original imbalanced data showed high precision (0.664) but poor recall (0.524), meaning the model was conservative, correctly identifying churners when it predicted churn, but missing nearly half of actual churners. SMOTE flipped this trade-off, achieving excellent recall (0.791) but lower precision (0.518), catching most churners but with more false alarms.

VAE latent features alone performed worst with F1 scores around 0.52, indicating substantial information was lost during compression. Even when combined with original features, VAE showed no improvement over using original features alone. This suggests the latent representations didn't discover new patterns but merely re-encoded existing information.

GAN synthetic data performed similarly to SMOTE with F1 score 0.594, despite being far more complex to generate. The sophisticated adversarial training produced no meaningful advantage over simple linear interpolation.

Table 1: Model Performance Comparison

Approach	Model	Precision	Recall	F1
Original	Logistic Regression	0.664	0.524	0.586
SMOTE	Logistic Regression	0.518	0.791	**0.627**
VAE-Latent	XGBoost	0.567	0.489	0.525
VAE-Combined	Logistic Regression	0.653	0.508	0.576
GAN-Synthetic	Logistic Regression	0.672	0.532	0.594

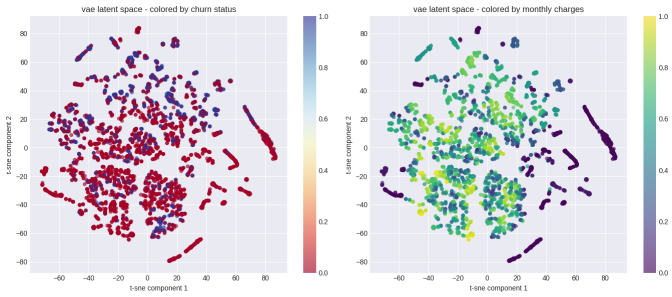

Figure 1: Latent Space Visualization

5.2 Training Dynamics

Both deep models required extensive optimization to achieve stable training. The VAE's reconstruction loss started at 25.6 and decreased to 17.0 after 500 epochs. While this 23% improvement seems significant, it translates to only 0.004 error per feature meaning good reconstruction but no classification benefit.

GAN training proved particularly challenging. Without our stabilization techniques, the discriminator quickly learned to perfectly identify all fakes, causing the generator to produce garbage. Our modifications achieved balanced competition with discriminator loss around 0.72 and generator loss around 1.87, indicating neither network dominated.

5.3 Feature Importance Analysis

An interesting insight came from the feature importance rankings. When combining VAE latent features with original features, the model slightly preferred latent features (29.2% vs 27.1%, ratio of 1.08). Howeeber, his didn't improve predictions, showing that even when models prefer latent features for their smooth, denoised properties, they're still just redundant encodings of the same information.

This shows that feature importance can be misleading. Models may prefer latent features for being smoother and denoised, but that doesn't mean they add predictive value. They're "mathematically" cleaner versions of the same information. The model preferred latent features because they're smoother and denoised, but they encode the same information as original features just in different form.

Traditional features maintained clear interpretability. Contract type, tenure, and monthly charges consistently ranked highest, aligning perfectly with intuition about what drives churn.

5.4 Latent Space Analysis

Visual inspection of the VAE latent space using t-SNE projection confirms our quantitative findings. Figure 1 shows the learned representations colored by churn status and monthly charges. The complete lack of separation between churned and non-churned customers demonstrates that the VAE optimized for reconstruction quality rather than class discrimination. While some structure emerges for continuous features like monthly charges (visible gradient patterns), this provides no benefit for binary classification. This visualization definitively shows why VAE features achieved only 0.525 F1 score - the latent space simply doesn't encode churn-relevant patterns.

6 Conclusion

This project provides comprehensive empirical evidence comparing deep generative models against traditional machine learning for telecom churn prediction. Our findings challenge prevailing assumptions about deep learning

superiority, demonstrating that SMOTE with Logistic Regression (F1: 0.627) consistently outperforms both VAE-enhanced features (F1: 0.576) and GAN-generated synthetic data (F1: 0.594).

6.1 Summary of Key Findings

Our investigation yielded three critical insights that have broad implications for applied machine learning. First, the VAE successfully learned to compress and reconstruct customer profiles with minimal error (0.004 per feature), yet these representations failed to improve classification. The conclusion we drew from this is that unsupervised representation learning optimizes for data compression, not class discrimination, at least in this case.

Second, our GAN implementation achieved stable adversarial training after careful tuning, with balanced losses (D: 0.72, G: 1.87) indicating proper adversarial dynamics. However, these sophisticated synthetic samples offered no advantage over SMOTE's simple linear interpolation. This suggests that for structured tabular data with clear feature semantics, complex generative models may be solving the wrong problem.

Third, VAE latent features dominated Random Forest splits (51.5%) without improving performance which reveals that high usage doesn't lead to unique contribution. The model preferred smooth, denoised latent features that ultimately encoded redundant information, teaching us that feature importance metrics can be misleading when features are correlated or redundant.

6.2 Practical Implications

For industry practitioners, our results provide clear guidance: when working with well-structured telecom datasets of moderate size (thousands to tens of thousands of samples), traditional methods remain optimal.

However, this doesn't mean deep generative models lack value entirely. They may prove beneficial in scenarios we didn't test: raw, unprocessed data where feature engineering hasn't been performed; truly massive datasets with millions of samples where deep learning's capacity becomes advantageous; temporal sequences requiring recurrent architectures; or multi-modal data combining tabular, text, and image information.

Our results also highlight the importance of aligning model objectives with task requirements. VAEs optimize the evidence lower bound, GANs optimize adversarial loss, but neither directly optimizes for class separation. This misalignment explains why these models can achieve their training objectives perfectly while failing at the downstream task.

6.3 Limitations and Future Directions

Several limitations warrant acknowledgment. Our dataset of 7,043 samples may be insufficient for deep learning methods to show their true potential. The binary classification task might be too simple to benefit from complex representations. Additionally, our VAE used unsupervised learning.

Future research should explore several promising directions. Supervised VAEs or adversarial autoencoders could directly optimize for discriminative representations. Temporal modeling using RNNs or Transformers could capture customer behavior evolution over time. Specialized architectures like CTGAN, TabNet, or NODE (Neural Oblivious Decision Trees) designed specifically for tabular data might bridge the performance gap. Finally, testing on much larger datasets (¿100,000 samples) could reveal whether deep learning eventually surpasses traditional methods given sufficient data.

6.4 Broader Impact

Beyond the specific application to churn prediction, this work offers valuable lessons for the machine learning community. It demonstrates the importance of rigorous baseline comparisons, without testing SMOTE, we might have incorrectly concluded that deep learning advances the state-of-the-art. It validates the principle that model complexity should match problem complexity, resisting the thought to apply cutting-edge methods where simpler solutions suffice.

Most importantly, this project exemplifies the value of negative results in research. Although we did our best to correctly implement and optimize our models, they still did not perform the best when compared against traditional methods. However, their failure to improve prediction performance isn't a failure of execution but might be finding about the nature of tabular data and the limits of current deep learning methods.

References

[1] Anonymous. Ctgan-enn: A tabular gan-based hybrid sampling method for imbalanced and overlapped data in customer churn prediction. *Preprint*, 2024.

[2] Anonymous. Synthetic tabular data generation using a vae-gan. *SSRN Preprint*, 2024.

[3] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.

[4] A. Idris, A. Khan, and D. Alghazzawi. Churn prediction in telecom with rf and pso-based balancing/feature selection. *Computers, Environment and Urban Systems*, 72:187–195, 2019.

[5] D. P. Kingma and M. Welling. Auto-encoding variational bayes. In *International Conference on Learning Representations (ICLR)*, 2014.

[6] T. Vafeiadis, K. Diamantaras, G. Sarigiannidis, and K. C. Chatzisavvas. A comparison of machine learning techniques for customer churn prediction. *Simulation Modelling Practice and Theory*, 55:1–9, 2015.

A Appendix

Including more context in this section to check for understanding and addressing any other challenges we may face. Would love some feedback on this if possible.

A.1 How the Models Work

The Variational Autoencoder (VAE) learns to represent customers in a smaller, compressed space where similar customers are located closer together. It then reconstructs the original data from this space, helping us identify key patterns in customer behavior. The Generative Adversarial Network (GAN) works differently—it trains two models at once: one that tries to generate realistic new customer records, and another that tries to tell real from fake. Over time, the generator becomes better at producing realistic examples that resemble true customer data.

A.2 What We Expect to Observe

At the beginning of training, the generated data may look random or unrealistic. As training continues, the models should begin to generate samples that closely match the real customer patterns, especially for the smaller churn class. The VAE's latent space should also show clusters of similar customers, helping visualize groups at higher risk of churn.

A.3 Handling Challenges

Working with tabular data is more difficult than images or text because the relationships between features are not spatial or sequential. If we encounter issues with training stability or poor synthetic results, we will explore specialized architectures like CTGAN or VAE–GAN, which are better suited for mixed numerical and categorical data. These models can serve as backup options to ensure realistic data generation.

A.4 Evaluating Model Results

Model evaluation will combine both numerical metrics and visual checks. We will compare prediction performance (Accuracy, Recall, F1-score) across models and visualize the latent space to see if customers naturally group into meaningful patterns. Improvements in minority class recall or more distinct clusters would indicate that the generative models are successfully learning the structure of churn behavior.